Friedrich-Alexander-Universität
Technische Fakultät

Advanced Deep Learning – Federated Learning and Differential Privacy

Linda-Sophie Schneider, Vincent Christlein

Pattern Recognition Lab,

Friedrich-Alexander-Universität Erlangen-Nürnberg WiSe 2025/26

Recap: A Conventional ML Pipeline

- **Collect & label data:** define target, sampling, and annotation rules.
- **Prepare data:** cleaning, preprocessing, feature extraction, splits.
- **Train model:** choose architecture, loss, optimizer, hyperparameters.
- **Evaluate:** validation, test set, metrics aligned with the task.
- **Deploy & monitor:** track drift, failures, and retrain when needed.

Key Point

The pipeline is repeatable, but **performance is fundamentally limited by the data**.

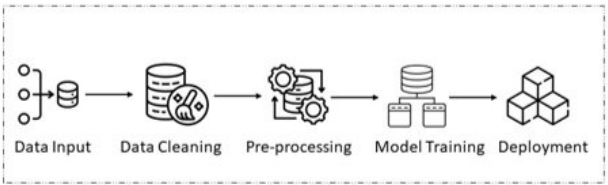


Image Source: https://www.design-reuse.com/article/61411-an-overview-of-machine-learning-pipeline-and-its-importance/

From Training Accuracy to Generalization

- Training minimizes **empirical risk** on observed samples; we care about **expected risk** on future data.
- Generalization improves when training data is **representative** of what we will see at test/deployment time.

Distribution View

$$\text{train data} \sim p_{\text{train}}(x, y) \quad \neq \quad \text{deployment data} \sim p_{\text{test}}(x, y)$$

If $p_{\text{train}} \neq p_{\text{test}}$ (dataset shift), **validation results can be misleading**.

Two Practical Drivers

- **Data quality:** noise, missing labels/features, inconsistent annotation, outliers.
- **Data distribution:** imbalance, rare subgroups, domain shift, covariate shift.

When Data Is Missing or Badly Distributed

The Need for Data

Modern models generalize best with **large, diverse, representative** datasets (e.g., across hospitals/devices).

The Data Paradox: Data Exists, But Is Hard to Centralize

- **Limited local data:** each site/device alone has too few samples.
- **Skew & missingness:** each silo covers only a slice (non-IID), with systematic gaps.
- **Data movement is constrained:** privacy/ownership, security risk, and bandwidth/cost.

Image Source: https://dev.to/vincajayi/how-to-deal-with-missing-data-15h2

What is Federated Learning (FL)?

Definition

Federated learning is a machine learning setting where multiple entities (clients) collaborate in solving a machine learning problem, under the coordination of a central server or service provider. Each client's raw data is stored locally and not exchanged or transferred; instead, focused updates intended for immediate aggregation are used to achieve the learning objective.

Definition proposed in [29]

Terminology:

- **Clients:** Devices or institutions with local data (e.g., smartphones, hospitals).
- **Server:** Coordinates training by sending model parameters and aggregating updates.
- **Global model:** The collaboratively-trained model across all clients.
- **Aggregation:** The process of combining model updates from clients to update the global model
- **Communication Rounds:** A single iteration of communication and model updates between the central server and the clients (e.g., one epoch of local training + communication)

Federated Learning: Comparision to Centralized Learning

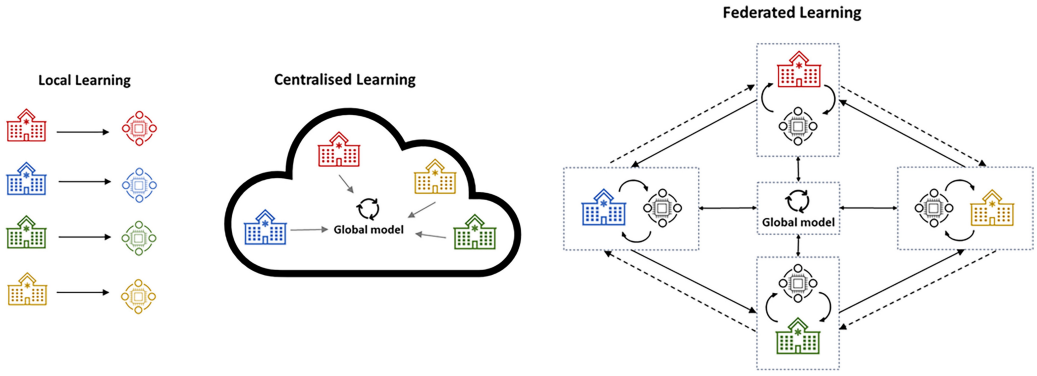


Image Source: [1]

Centralized to Decentralized FL

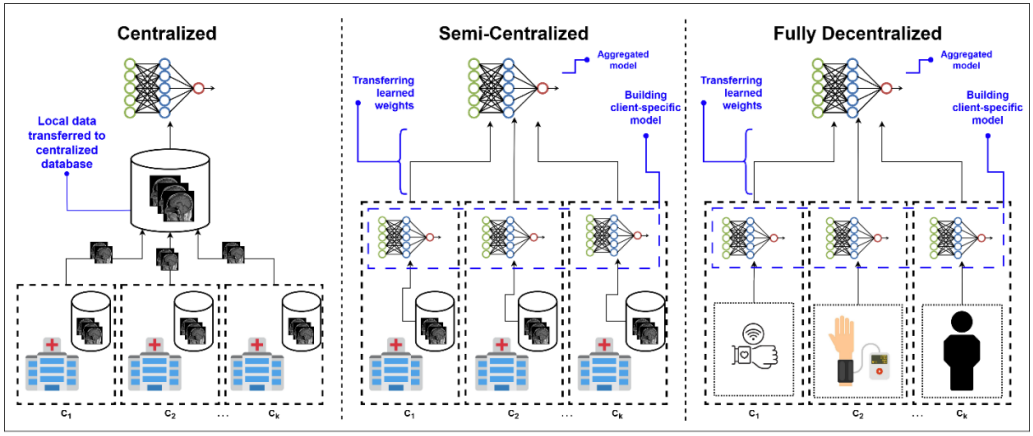


Image Source: [30]

Centralized to Decentralized FL

Centralized Learning:

- Data from all clients is collected and stored on a central server.
- The model is trained on this centralized dataset.
- Requires data transfer, raising privacy and security concerns.

Semi-centralized Federated Learning:

- Each client keeps its data locally and trains client-specific models.
- The server aggregates model updates from clients to form a global model.
- Reduces data transfer, enhancing privacy.

Decentralized Federated Learning:

- Clients (individuals) maintain their own data and local networks.
- No central server; clients communicate directly with each other.
- Maximizes privacy but increases communication complexity.

The Math behind FL

The FL Setup: Intuition

The Intuition: Imagine a team writing a book where no one is allowed to read each other's draft chapters (Data Privacy). They only submit their edits (Updates) to an editor (Server).

The Players & Variables

- **Server:** The coordinator. Holds the **global model** $w \in \mathbb{R}^d$.
- **Clients (K):** The workers (e.g., hospitals). Each client k holds a private dataset $\mathcal{D}_k$ with n_k samples.
- **Total Data:** $n = \sum_{k=1}^{K} n_k$ is the total number of samples across the world.
-
- **Relative Weight (p_k):** $p_k = \frac{n_k}{n}$. Clients with more data get a louder "vote."

The FL Setup: Goal

The Goal: Global Optimization

We want to find a model w that minimizes the **global loss** $F(w)$, which is the weighted average of valid local losses:

$$\min_w F(w) = \sum_{k=1}^{K} \underbrace{\frac{n_k}{n}}_{\text{Weight } p_k} \cdot \underbrace{F_k(w)}_{\text{Local Loss}}$$

Client k: The Biased Local View

Each client acts like it is training a normal model, but it has a "blind spot"—it only sees its own data.

Local Empirical Risk $F_k(w)$

Client k calculates the average error on its own n_k samples:

$$F_k(w) = \frac{1}{n_k} \sum_{(x,y)\in\mathcal{D}_k} \ell\big(\underbrace{f_w(x)}_{\text{Prediction}}, \underbrace{y}_{\text{Label}}\big)$$

- **The Heterogeneity Problem (Non-IID):** Client A might have only "sick" patients, while Client B has "healthy" ones.
- **The Conflict:** The model that is best for Client A ($w_A^\star$) is **not** the best model for the world ($w^\star$).
- **Intuition:** If you only see cats, you will think the whole world is cats. The server's job is to correct this bias.

The Solution: FedAvg

Proposed by McMahan et al. [28], **Federated Averaging (FedAvg)** is the standard iterative loop to solve this.

The Protocol (Round t):

1. **Selection:** Server picks a random fraction of clients S_t.
2. **Broadcast:** Server sends the current global model w_t to them.
3. **Local Training (The "Work"):** Client k initializes with w_t and runs SGD for E epochs using learning rate η:
$$w_{t+1}^k \leftarrow \text{SGD}(w_t, \mathcal{D}_k)$$
4. **Aggregation (The "Consensus"):** Server averages the resulting models to update the global state:
$$w_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{n_{total}} w_{t+1}^k$$

FedAvg Detail: The Local Update

Inside Client k: Instead of sending gradients (like distributed SGD), we perform significant local computation (E epochs).

- **Initialization:** $w \leftarrow w_t$ (server weights)
- **Loop:** For epoch $e = 1 \dots E$, for batch $b \in \mathcal{D}_k$:

$$w \leftarrow w - \eta \nabla F_k(w; b)$$

- **Output:** Send final weights w_{t+1}^k (or delta Δ_k) back to server.

Trade-off

Pros: High computation per communication round (saves bandwidth).
Cons: If E is large, the model walks far away from w_t towards its *local* optimum $w_k^\star$.

FedAvg: The Aggregation Step

Once the selected clients upload their results, the server must combine them. This is not a simple average; it is a **Weighted Average**.

The Logic: Weighted Voting

The server computes the new global model w_{t+1} by weighing each client's contribution based on how much data they possess.

$$w_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{n_{S_t}} w_{t+1}^k$$

- n_k **(Client Data):** The number of samples on client k.
- n_{S_t} **(Round Data):** The sum of samples of all *active* clients ($\sum n_k$).
- **Intuition:** A hospital with 10,000 patients gets a "louder vote" than a clinic with 10 patients. This ensures the model learns from the true data distribution, not just the number of devices.

5. Advanced Implementation: Delta Aggregation

In modern frameworks (and Secure Aggregation), clients rarely send the full model parameters. They send the **Updates** (Deltas).

Sending the "Direction" (Δw)

Instead of sending the new location (w_{t+1}^k), the client sends the vector of movement:

$$\Delta w_{t+1}^k = \underbrace{w_{t+1}^k}_{\text{New Local}} - \underbrace{w_t}_{\text{Global Start}} \tag{1}$$

Why use Deltas?

- **Efficiency:** It is easier to compress small changes (many values might be zero/sparse).
- **Server Optimization (FedOpt):** The server treats the aggregated Δ as a **"Pseudo-Gradient"**. Instead of just applying it, the server can use **FedAdam** or **FedYogi** to accelerate training using server-side momentum.

The Challenge: Non-IID Data & Client Drift

The Reality

Data is **Non-IID**. Client A has cats, Client B has dogs.

$$\nabla F_A(w) \neq \nabla F_B(w)$$

- Because we run multiple local steps ($E > 1$), Client A moves towards "Cat Optimum" and Client B towards "Dog Optimum".
- **Client Drift:** The average of these divergent models may not lie on the path to the true global optimum.
- *Result: Slow convergence and unstable accuracy.*

Client Drift

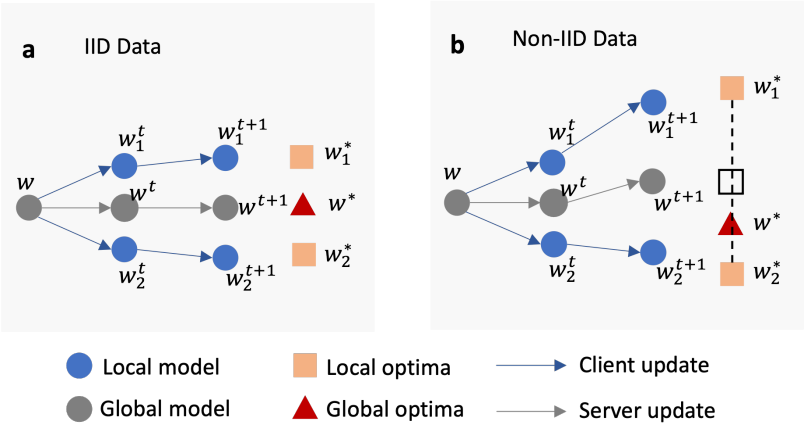


Image Source: [31]

Addressing Drift: FedProx

FedProx [24] is a direct modification of FedAvg to fix drift.

Idea: Constrain the Local Training

Don't let the client wander too far from the server's starting point w_t.

Modified Local Objective

$$\min_w h_k(w; w_t) = \underbrace{F_k(w)}_{\text{Local Loss}} + \underbrace{\frac{\mu}{2}\|w - w_t\|^2}_{\text{Proximal Term}}$$

- **Effect:** If the local gradient tries to pull w far away, the proximal term pulls it back towards w_t.
- **Heterogeneity:** Safely allows for variable local epochs (stragglers) because updates are dampened.

Concept Check: The Physics of FedProx

We defined the FedProx objective as:

$$\min_w F_k(w) + \underbrace{\frac{\mu}{2}\|w - w_t\|^2}_{\text{Proximal Term}}$$

Scenario: Imagine your local model w has drifted far away from the global server model w_t during training.
Question: Physically, what does the proximal term do to the update vector?

(A) It pushes w even further in the direction of the local gradient (Momentum).

(B) It acts like a **rubber band**, pulling w back towards w_t.

(C) It forces w to move orthogonal to the gradient.

Answer: The "Rubber Band" Effect

Correct Answer: (B)

The Mathematics

The gradient of the proximal term is:

$$\nabla \left(\frac{\mu}{2} \|w - w_t\|^2 \right) = \mu(w - w_t)$$

- In Gradient Descent ($w \leftarrow w - \eta \nabla L$), we move **against** the gradient.
- The update term becomes $-\eta\mu(w - w_t) = \eta\mu(w_t - w)$.
- **Interpretation:** This is a vector pointing from your current spot w directly back to the server weights w_t.

Takeaway

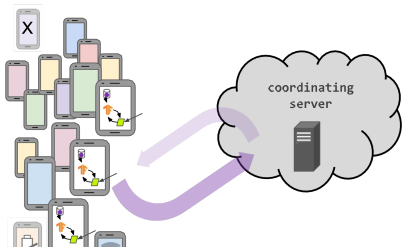
The larger the drift ($|w - w_t|$) or the penalty (μ), the stronger the "rubber band" pulls you back to safety.

FL Types

Cross-Device FL vs. Cross-Silo FL

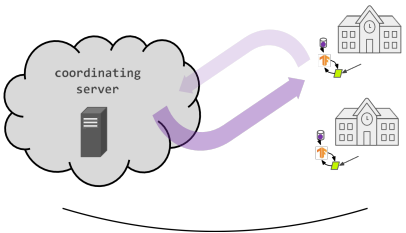
Cross-Device

millions of intermittently available client devices



Cross-Silo

small number of clients (institutions, data silos), high availability



Cross-Device FL vs. Cross-Silo FL

Aspect	Cross-Device Federated Learning	Cross-Silo Federated Learning
Participants	Up to millions of devices (e.g., smartphones, IoT)	Few organizations (e.g., hospitals, banks)
Data Distribution	Highly diverse across devices	More structured and more aligned
Device Availability	Unreliable (devices may drop out)	Reliable and consistent participation
Computational Power	Limited (edge devices with low resources)	High (dedicated servers or cloud infrastructure)
Aggregation	Based on gradients (due to low computation power available)	Based on model parameters (due to high computation power available)
Applications	Mobile apps, IoT, personalized services	Healthcare, finance, shared organizational tasks

Synchronous FL vs. Asynchronous FL

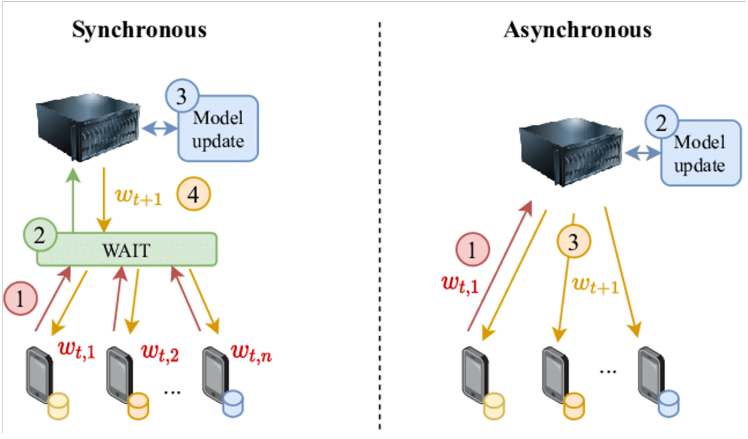


Image Source: [32]

Synchronous FL vs. Asynchronous FL

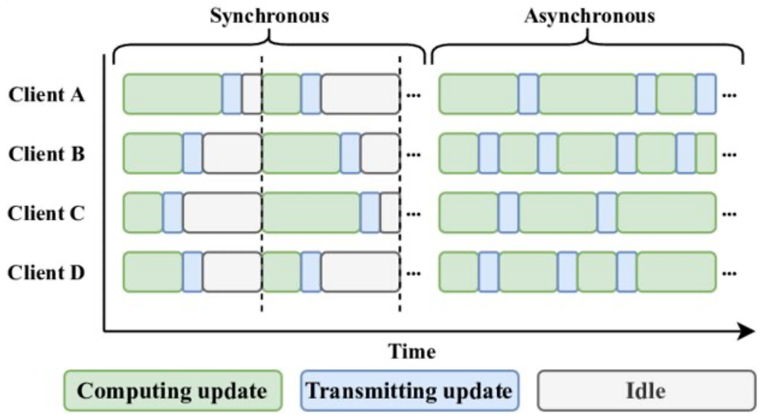


Image Source: [33]

Synchronous FL vs. Asynchronous FL

Aspect	Synchronous FL	Asynchronous FL
Update Timing	All updates occur in fixed communication rounds	Updates are processed as they arrive
Coordination	Requires strict coordination; all participants must complete updates for each round	No coordination required; participants work independently
Fault Tolerance	Sensitive to stragglers (slow participants delay rounds)	Tolerant to device dropouts or delays
Scalability	Limited due to synchronization requirements	Highly scalable, suitable for large-scale systems
Consistency	Consistent global updates with contributions from all in each round	Model may receive stale updates, requiring robust algorithms to ensure consistency
Efficiency	Inefficient for dynamic or large-scale environments due to idle waiting time	Efficient use of resources as devices operate at their own pace
Example Use Case	Cross-silo FL	Cross-device FL

Challenges in FL

Challenges in FL

Structural Challenges

Challenge: Real-world FL in healthcare is not easy to implement.

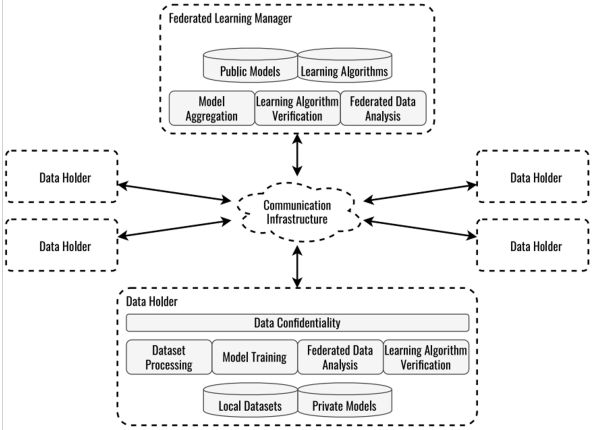


Image Source: [25]

Challenges in FL

Non-IID Data

Challenge: Real-world FL in healthcare is always non-IID

Different label distribution, different imaging machines, different labeling protocols, different clinicians, different demographics (race, age, gender, etc.), different data size, etc.

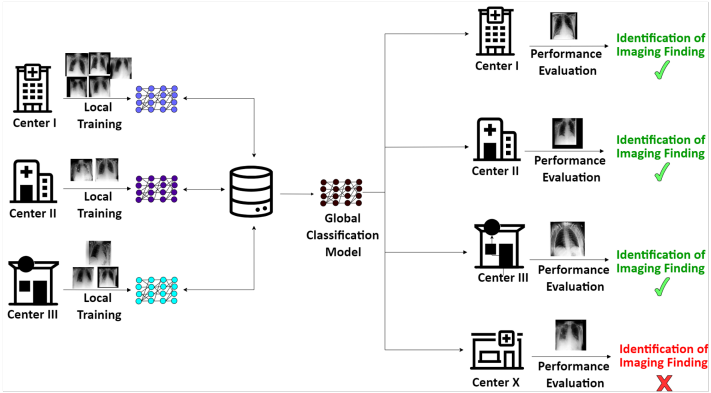


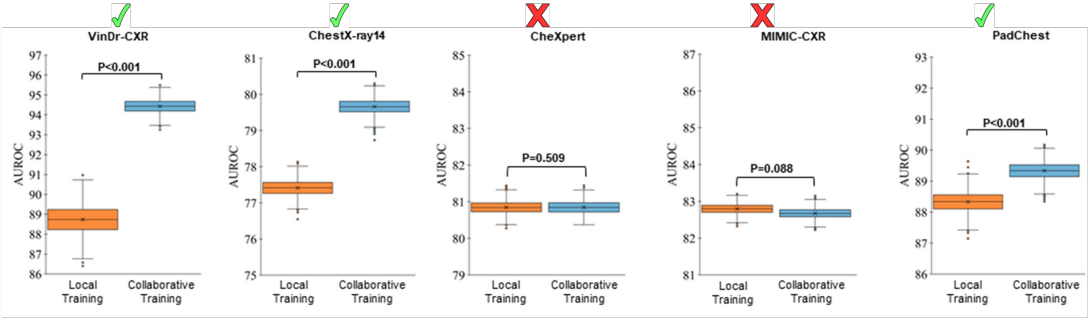
Image Source: [18]

Challenges in FL

Non-IID Data

Challenge: Real-world FL in healthcare is always non-IID

► Clients with large enough and representative data won't benefit from FL anymore.



Data Sources (all publicly available for research): Nguyen et al. VinDr-CXR: An open dataset of chest X-rays with radiologist's annotations. Sci. Data (2022) | Wang et al. ChestX-ray8: Hospital-scale chest X-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases, 2017 CVPR | Irvin et al. CheXpert: A large chest radiograph dataset with uncertainty labels and expert comparison. AAAI 33 (2019) | Johnson et al. MIMIC-CXR, a de-identified publicly available database of chest radiographs with free-text reports. Sci. Data (2019) | Bustos et al. PadChest: A large chest x-ray image dataset with multi-label annotated reports. Med. Image Anal. (2020)

Challenges in FL

Different Ground Truth

Challenge: Not always data with similar ground truth available across different clients

Three separate data centers intend to train AI models for the prediction of different diseases. Conventional federated learning: only overlapping objectives can collaborate on training a neural network for the detection of cardiomegaly.

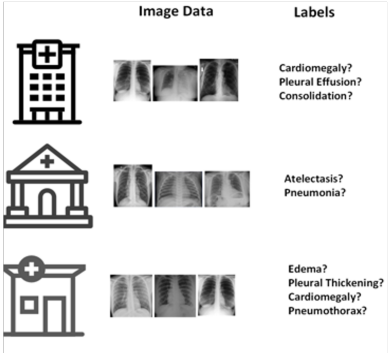


Image Source: [17]

Challenges in FL

Different Ground Truth

Challenge: Not always data with similar ground truth available across different clients

► Solution: Flexible federated learning approaches that can leverage heterogeneous labels across clients.

All centers collaborate to train a common backbone network and individual classification heads using all their data.
For classification, each center employs the common backbone and the local classification head.

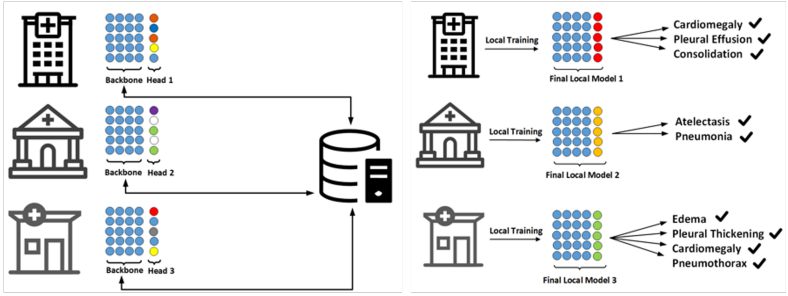


Image Source: [17]

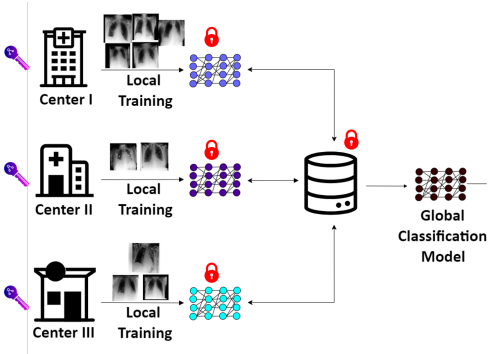
Challenges in FL

Un-trusted Server

Challenge: What if we don't trust the server? It still has access to the individual parameters/gradients

Homomorphic encryption

- Each client encrypts all the parameters before transmission to the server.
- All the aggregation happens in the encrypted domain.
- Server won't have access to any meaningful information.
- But the clients know the decryption code!



Image

Source: [16]

Homomorphic Encryption (HE): General Definition

Definition

Homomorphic Encryption is a cryptographic technique that enables computations to be executed directly on encrypted data without requiring prior decryption.

- **Core Concept:** It allows a third party to process data while it remains in its encrypted state. The result of operations performed on the ciphertext, when decrypted, matches the result as if the operations had been performed on the plaintext.
- **Mathematical Property:** If $Enc(x)$ is the encryption of x, HE supports operations $\oplus$ such that:

$$Dec(Enc(x) \oplus Enc(y)) = x + y \quad \text{(Additive)}$$

Homomorphic Encryption (HE): Key Properties

Key Properties & Challenges

- **End-to-End Privacy:** Unlike standard encryption (which protects data at rest or in transit), HE protects data *during processing*.
- **Schemes:** Popular schemes include CKKS, which allows for encrypted arithmetic on approximate numbers (real values).
- **Overhead:** HE introduces significant computational overhead and larger ciphertext message sizes compared to plaintext operations, which can limit scalability in resource-constrained environments.

The SPDZ Algorithm: Additive Secret Sharing

The SPDZ protocol (Damgård, Pastro, Smart, Zakarias) secures computation via **additive secret sharing**.

1. **Setup and Encoding**
   - **Large Prime Field:** A large prime Q is generated by the crypto provider.
   - **Fixed-Point Arithmetic:** Real-valued weights w are encoded into integers x.
2. **Secret Sharing Scheme**
   To encrypt a secret x across N sites:
   - Sites 1 to $N-1$ generate random integers $x_i \in (0, Q)$.
   - The last site N calculates its share to satisfy the sum:

$$x_N = \left(x - \sum_{i=1}^{N-1} x_i \right) \pmod{Q}$$

No single site holds the value x; it exists only as the sum of distributed shares.

Aggregation in the Encrypted Domain

Homomorphic Property (Additivity)

Because the secret sharing is linear, the server can sum the encrypted shares directly. If client A holds share $[w]_A$ and client B holds $[w]_B$:

$$\text{Aggregated Share} = ([w]_A + [w]_B) \pmod{Q}$$

The sum of the shares equals the share of the sums. The server computes the global model update without decrypting inputs.

Decryption (Reconstruction)

Once the server computes the aggregated encrypted shares, the result is sent back to the clients.

$$x_{\text{global}} = \left(\sum_{i=1}^{N} \text{Share}_i \right) \pmod{Q}$$

Clients combine their results to reveal the new global model parameters.

Challenges in FL

Data Privacy

Question: Is FL alone enough to guarantee the privacy and release an AI model, trained with private data?

- No, because the server still has access to the individual parameters/gradients.
- Those parameters/gradients can be exploited to reconstruct the original data.

→ We need additional privacy-preserving techniques like Differential Privacy (DP)!

Image Source: https://www.msn.com/en-us/money/other/personal-data-of-100-million-americans-leaked-in-major-cybersecurity-incident/ss-AA1rAbUg

Gradient Inversion Attacks

Gradient Inversion

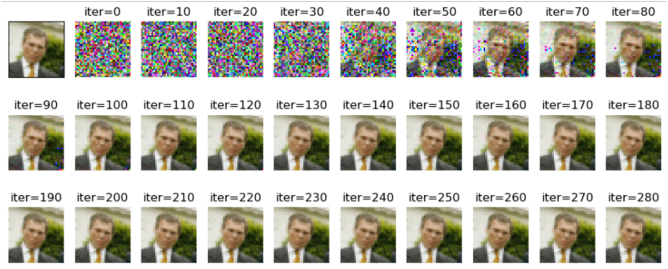


Image Source: [13]

Gradient Inversion

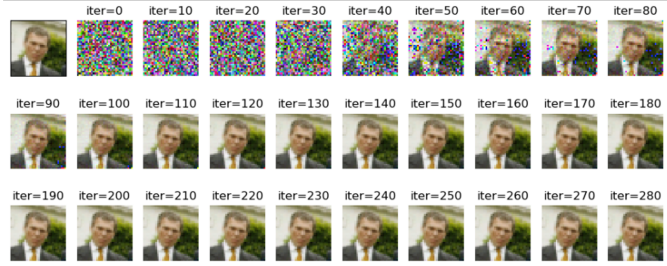


Image Source: [13]

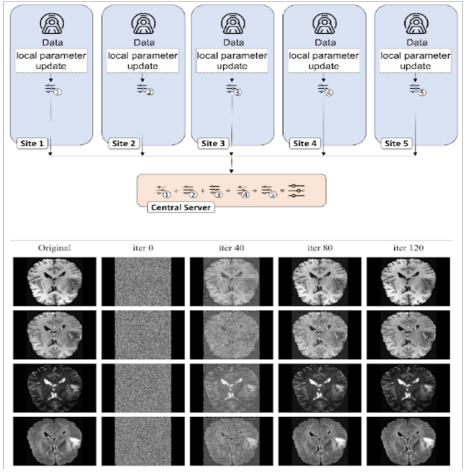


Image Source: [15]

Gradient Inversion Attacks

Idea: An adversary with access to model gradients can reconstruct the original training data by optimizing a dummy input to match the observed gradients.

Mathematical Formulation

Given:

- Observed gradient: $g = \nabla_w F(w; x, y)$
- Dummy input: $\tilde{x}$ with label $\tilde{y}$

The attacker solves:

$$\min_{\tilde{x},\tilde{y}} \|\nabla_w F(w; \tilde{x}, \tilde{y}) - g\|^2$$

- By iteratively updating $\tilde{x}$ and $\tilde{y}$, the attacker can recover inputs that produce gradients similar to g.
- This exposes sensitive information about the original training data.

Improved Deep Leakage (iDLG): Analytic Label Extraction

Problem with DLG: Optimizing for both input $\tilde{x}$ and label $\tilde{y}$ is unstable and slow.

Insight: Gradients Leak Labels Analytically

For Cross-Entropy loss $\mathcal{L} = -\sum y_i \log p_i$, the gradient w.r.t. the last layer logit z_i is:

$$\frac{\partial \mathcal{L}}{\partial z_i} = p_i - y_i \tag{2}$$

- Since y is one-hot, only the true class index c has $y_c = 1$.
- Since $p_i \in (0, 1)$, the gradient $p_c - 1$ is **always negative** for the true class.

Result: The attacker determines $\tilde{y}$ instantly by checking gradient signs, reducing the problem to finding $\tilde{x}$ only:

$$\tilde{x}^* = \arg\min_{\tilde{x}} \|\nabla_w F(w; \tilde{x}, \mathbf{y_{true}}) - g\|^2$$

Limitations: The Impact of Batch Size

Gradient inversion is an ill-posed inverse problem. The difficulty depends heavily on the **Batch Size (B)**.

- **Small Batch ($B = 1$):** The shared gradient g corresponds to a single image. Reconstruction is highly accurate.
- **Large Batch ($B > 8$):** The shared gradient is a sum (or average):

$$g = \frac{1}{B} \sum_{i=1}^{B} \nabla \ell(w; x_i, y_i)$$

- **Consequence:** The attacker must solve for B unknown images from a single gradient vector.
- As B increases, the reconstruction quality drops and images become indistinguishable noise.

Note: Secure Aggregation acts as a "virtual large batch" summing updates from all clients, further mitigating this attack.

The Math behind Differential Privacy

Differential Privacy: The Intuition

Before the math, let's understand the goal.

The "Opt-Out" Scenario

Imagine two universes:

- **Universe A (Dataset D):** Includes your sensitive medical record.
- **Universe B (Dataset D'):** Identical to D, but **without** your record.

The Guarantee: Differential Privacy ensures that an attacker looking at the model's output cannot tell which universe they are in.

- If the output probabilities are nearly identical for D and D', the attacker learns nothing specific about you.
- *"Privacy isn't subjective; it's a quantitative limit on the influence of any single data point."*

Formal Definition: (ε, δ)-DP

We quantify "indistinguishable" using two parameters:

Definition

A randomized algorithm $\mathcal{M}$ is (ε, δ)-DP if for all neighboring datasets D, D' (differing by one record) and all outputs S:

$$\Pr[\mathcal{M}(D) \in S] \leq e^{\varepsilon} \cdot \Pr[\mathcal{M}(D') \in S] + \delta \tag{3}$$

- ε **(Privacy Budget):** The maximum allowed multiplicative difference in outcome probability.
  - Lower ε (e.g., 1.0) $\rightarrow$ High Privacy (Outputs look the same).
  - Higher ε (e.g., 10.0) $\rightarrow$ Low Privacy (Outputs diverge).
- δ **(Failure Probability):** The tiny probability (usually $< \mathbf{1}/N$) that the bound fails entirely. We allow a small chance of failure to make the math work for Gaussian noise.

The Mechanism: Sensitivity and Noise

How do we ensure the output doesn't change much when one person is removed? We add noise proportional to the **Sensitivity**.

Sensitivity (Δf)

The maximum amount the function output $f(D)$ can change if one person is removed.

$$\Delta f = \max_{D,D'} \|f(D) - f(D')\|_{\mathbf{2}}$$

- **High Sensitivity:** One person can drastically change the result $\rightarrow$ Needs **more noise** to hide them.
- **The Gaussian Mechanism:** Add noise $\mathcal{N}(\mathbf{0}, \sigma^{\mathbf{2}})$ where $\sigma \propto \frac{\Delta f}{\varepsilon}$ [23].

DP-SGD: Privacy in Deep Learning

In Deep Learning, we can't calculate sensitivity easily because gradients are unbounded. **DP-SGD** (Abadi et al., 2016) solves this in two steps:

1. **Gradient Clipping (Bounding Sensitivity):** Since we don't know how large a gradient g_i might be, we force it to be small.

$$\bar{g}_i = g_i / \max\left(\mathbf{1}, \frac{\|g_i\|_2}{C}\right)$$

   Now, no single update can have a norm larger than C. Sensitivity is now exactly C.

2. **Noise Injection:** We sum the clipped gradients and add Gaussian noise scaled to C:

$$\tilde{g} = \sum \bar{g}_i + \mathcal{N}(\mathbf{0}, \sigma^2 C^2 \mathbf{I})$$

The model updates using this noisy, clipped gradient $\tilde{g}$.

Privacy Accounting: The "Budget"

Every time we update the weights (step t), we "spend" a little bit of our privacy budget ε.

The Problem of Composition

Deep learning takes thousands of steps (T). If we simply add up the ε from every step, the total ε becomes huge (useless privacy).

The Solution: Advanced Accounting We use **Rényi DP** or **Moments Accountant**.

- **Intuition:** It's like a "bulk discount" for privacy. The worst-case leakage rarely happens at every single step.
- **Subsampling Amplification:** Because we only use a random mini-batch (fraction q) at each step, privacy is amplified. If your data wasn't in the batch, you leaked nothing.

Gaussian Differential Privacy (GDP)

A modern, intuitive framework (Dong et al., 2019) views privacy as a **Hypothesis Testing** problem.

- **The Game:** An attacker tries to distinguish between two distributions: P (with your data) and Q (without your data).
- **Definition:** An algorithm is μ-GDP if distinguishing P from Q is as hard as distinguishing two Gaussians $\mathcal{N}(\mathbf{0}, \mathbf{1})$ and $\mathcal{N}(\mu, \mathbf{1})$.

Why use GDP?

- **Central Limit Theorem:** Just as sum of random variables converges to a Gaussian, the privacy loss of many training steps converges to GDP.
- It provides exact composition theorems, often yielding tighter bounds than (ε, δ) for deep learning.

DP in Deep Learning

In simple terms

► Add calibrated noise while clipping the gradients!

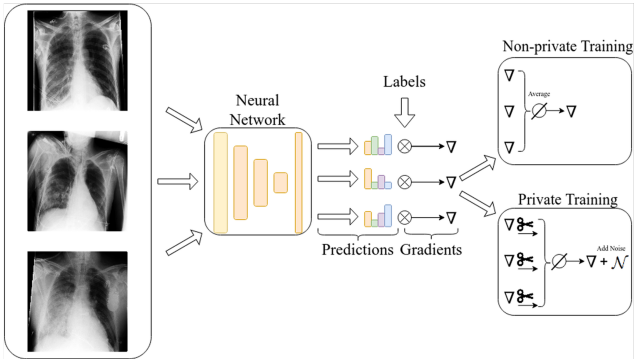


Image Source: [10]

Challenges in DP

Challenges in Implementing DP in Deep Learning

Core Issue: The privacy-preserving process inevitably affects the model's performance due to the constant addition of noise during training.

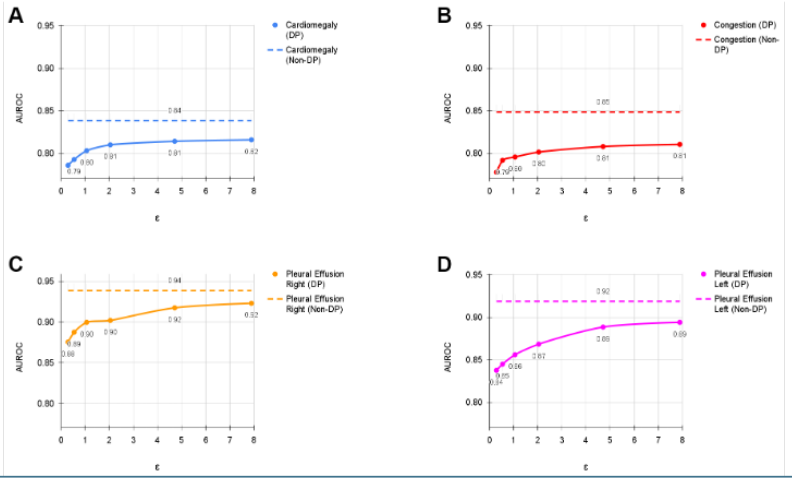
Major Trade-offs

- **Privacy-Utility Trade-off:** As privacy guarantees increase (i.e., **lower** ε values), the model's utility (e.g., Accuracy, F1-Score) typically decreases.
  - *Note:* Strict privacy ($\varepsilon \approx \mathbf{1}$) often leads to significant degradation, whereas moderate budgets ($\varepsilon \approx \mathbf{10}$) may preserve utility.
- **Privacy-Fairness Trade-off:** With increasing privacy (lower ε values), the model may disproportionately discriminate against underrepresented subgroups in the training data.
  - Noise and clipping can suppress the signals of minority classes, exacerbating disparate impact.

Challenges in DP

Privacy-Utility Trade-off

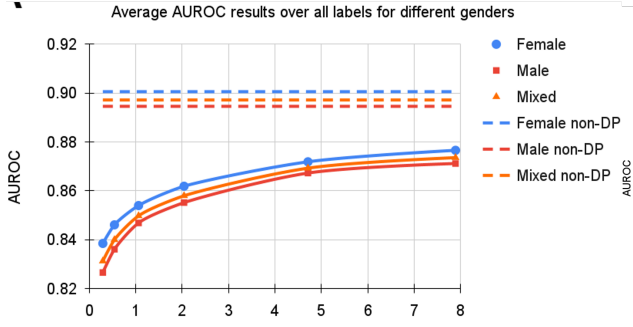
Example: Multilabel classification of different pulmonary diseases using chest x-rays.



Challenges in DP

Privacy-fairness Trade-off

Privacy-fairness trade-off is a complex issue that varies by application domain.



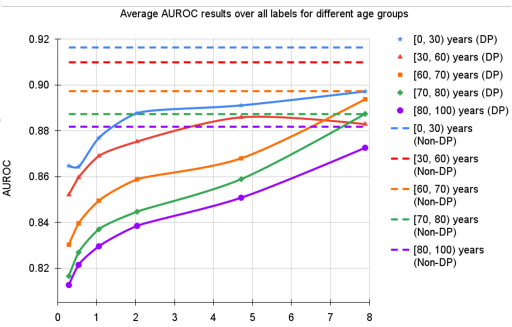


Image Source: [8]

Challenges in DP

Further Challenges

Privacy-fairness trade-off is a complex issue that varies by application domain.
There are other significant challenges with DP in deep learning:

- **Batch Normalization is incompatible:**
  It relies on batch-level statistics, which destroys the independence required for privacy analysis and can leak information. *Solution:* **Group Normalization** is used instead.
- **Large models are not completely effective yet:**
  Architectures like Vision Transformers (ViTs) are highly sensitive to gradient noise. They often require excessive noise to maintain privacy guarantees, which severely degrades utility compared to CNNs.
- **Data augmentation is problematic:**
  It generally increases gradient variance and can create dependencies between samples, complicating privacy accounting. It is often avoided in DP training.
- **Hyperparameter Sensitivity:**
  Choices for batch size, learning rate (often needs to be larger), and optimizers have substantially larger effects on convergence in DP training compared to conventional training.

Challenges in DP: Practical Recommendations

Tips for improving the Privacy-Utility Trade-off:

- **Pretraining is essential:** Training from scratch with DP is difficult due to noise. Pretraining on large, representative (public or self-supervised) datasets is practically mandatory to ensure convergence and utility.
- **Leverage Large Datasets:** The utility cost of DP diminishes as dataset size increases. Large-scale data can mitigate trade-offs to a great extent, making $\varepsilon \approx \mathbf{10}$ viable for clinical accuracy.
- **Optimizer Choice:** The **NAdam** optimizer is often recommended as the first choice for DP training to handle noisy gradients effectively.
- **Avoid Data Augmentation:** Unlike standard DL, augmentation increases gradient variance, which exacerbates instability in DP training. It is often best to disable it.
- **Set δ Correctly:** Choose a fixed $\delta \ll \mathbf{1}/n$ (smaller than the inverse of the training set size) to satisfy the theoretical requirements of DP.
- **Increased Training Time:** DP-SGD requires significantly more epochs to converge compared to non-private training (up to $\mathbf{10}\times$ longer) due to the noise injection.
- **Hyperparameter Tuning: Larger learning rates** are typically required for DP training compared to standard training to overcome the noise floor.

Summary: FL + DP for Medical AI

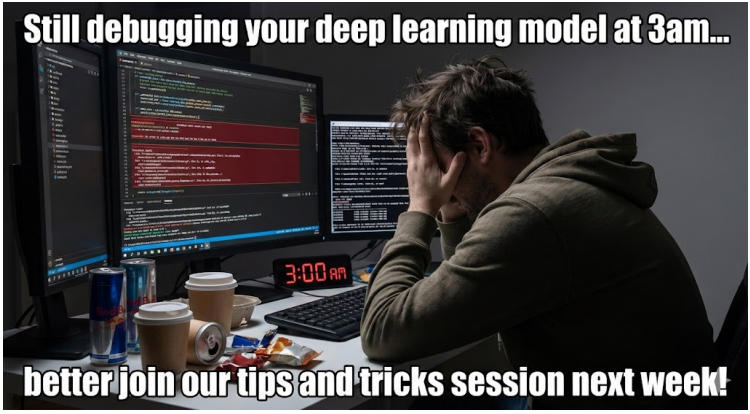
- **Federated Learning** breaks down data silos by enabling collaborative training across institutions without sharing raw patient data, aligning with regulations.
- **FL Challenges:** Data heterogeneity (**Non-IID**) causes client drift, requiring specialized aggregation strategies like **FedProx** to ensure convergence.
- **The Privacy Gap:** FL alone is insufficient; **Gradient Inversion Attacks** can reconstruct pixel-perfect medical images from shared model updates.
- **Differential Privacy** offers the gold standard defense by clipping gradients and injecting calibrated noise (DP-SGD), ensuring the model output is invariant to any single patient's data.
- **DP Challenges:** The inherent **privacy-utility trade-off** requires careful hyperparameter tuning, pretraining, and often larger datasets to maintain clinical-grade performance.
- **The Reality:** While trade-offs exist, recent large-scale studies confirm that FL + DP can achieve clinical-grade accuracy (e.g., in Chest X-rays) at moderate privacy budgets ($\epsilon \approx$ **10**).

NEXT TIME

ADVANCED

ON DEEP LEARNING

Tips and Tricks



Comprehensive Questions

1. Distinguish between *Cross-Silo* and *Cross-Device* FL. How does Non-IID data distribution cause "client drift" in standard Federated Averaging (FedAvg)?
2. Explain the mechanism of *Gradient Inversion Attacks* (e.g., DLG, iDLG). Why does using a small Batch Size ($B = \mathbf{1}$) make data reconstruction significantly easier for an adversary?
3. In the DP-SGD algorithm, what are the specific roles of *Gradient Clipping* and *Noise Injection*? How does reducing the Privacy Budget (ε) impact the noise magnitude and model utility?
4. Define *Homomorphic Encryption (HE)* in the context of FL. How does the additive property of HE allow a server to aggregate model updates without ever decrypting them?
5. What are the *primary challenges* when implementing Differential Privacy in deep learning, particularly regarding model architecture and training stability?

References

References I

[1] Zhen Ling Teo, Liyuan Jin, Nan Liu, Siqi Li, Di Miao, Xiaoman Zhang, Wei Yan Ng, Ting Fang Tan, Deborah Meixuan Lee, Kai Jie Chua, John Heng, Yong Liu, Rick Siow Mong Goh, and Daniel Shu Wei Ting. "Federated machine learning in healthcare: A systematic review on clinical applications and technical architecture". In: *Cell Reports Medicine* 5.2 (Feb. 2024), p. 101419.

[2] Zhiqi Bu, Jinshuo Dong, Qi Long, and Su Weijie. "Deep Learning with Gaussian Differential Privacy". In: *Harvard Data Science Review* (July 2020).

[3] Larry Wasserman and Shuheng Zhou. "A Statistical Framework for Differential Privacy". In: *Journal of the American Statistical Association* 105.489 (Mar. 2010), pp. 375–389.

[4] Tom Farrand, Fatemehsadat Mireshghallah, Sahib Singh, and Andrew Trask. *Neither Private Nor Fair*. Tech. rep. ACM, Nov. 2020, pp. 15–19.

[5] Eugene Bagdasarian and Vitaly Shmatikov. "Differential Privacy Has Disparate Impact on Model Accuracy". In: *Neural Information Processing Systems* abs/1905.12101 (2019).

References II

[6] Martin Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. *Deep Learning with Differential Privacy*. Tech. rep. ACM, Oct. 2016, pp. 308–318.

[7] Georgios Kaissis, Alexander Ziller, Jonathan Passerat-Palmbach, Théo Ryffel, Dmitrii Usynin, Andrew Trask, Ionésio Lima, Jason Mancuso, Friederike Jungmann, Marc-Matthias Steinborn, Andreas Saleh, Marcus Makowski, Daniel Rueckert, and Rickmer Braren. "End-to-end privacy preserving deep learning on multi-institutional medical imaging". In: *Nature Machine Intelligence* 3.6 (May 2021), pp. 473–484.

[8] Soroosh Tayebi Arasteh, Alexander Ziller, Christiane Kuhl, Marcus Makowski, Sven Nebelung, Rickmer Braren, Daniel Rueckert, Daniel Truhn, and Georgios Kaissis. "Preserving fairness and diagnostic accuracy in private large-scale AI models for medical imaging". In: *Communications Medicine* 4.1 (Mar. 2024).

[9] Cynthia Dwork and Aaron Roth. "The Algorithmic Foundations of Differential Privacy". In: *Foundations and Trends® in Theoretical Computer Science* 9.3-4 (Aug. 2014), pp. 211–487.

References III

[10] Soroosh Tayebi Arasteh, Mahshad Lotfinia, Paula Andrea Perez-Toro, Tomas Arias-Vergara, Mahtab Ranji, Juan Rafael Orozco-Arroyave, Maria Schuster, Andreas Maier, and Seung Hee Yang. "Differential privacy enables fair and accurate AI-based analysis of speech disorders while protecting patient data". In: *npj Artificial Intelligence* 1.1 (Nov. 2025).

[11] Samin Dehbashi Sani and Tara Salman. *iDLG-IG: an Explainability Guided Model Inversion Attack*. Tech. rep. IEEE, Sept. 2025, pp. 1–6.

[12] Desiree Xavier, Luiz Leite, André Riker, and Eirini Eleni Tsilopoulou. *Federated Learning Attack: Fast Image Leakage from Gradients*. Tech. rep. IEEE, Oct. 2025, pp. 1–6.

[13] Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. "iDLG: Improved Deep Leakage from Gradients". In: *arXiv.org* abs/2001.02610 (2020).

[14] Alexei Baevski, Henry Zhou, rahman Abdel-Mohamed, and Michael Auli. "wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations". In: *Neural Information Processing Systems* abs/2006.11477 (2020).

References IV

[15] Soroosh Tayebi Arasteh, Cristian David Ríos-Urrego, Elmar Nöth, Andreas Maier, Seung Hee Yang, Jan Rusz, and Juan Rafael Orozco-Arroyave. *Federated Learning for Secure Development of AI Models for Parkinson's Disease Detection Using Speech from Different Languages*. Tech. rep. ISCA, Aug. 2023, pp. 5003–5007.

[16] Daniel Truhn, Soroosh Tayebi Arasteh, Oliver Lester Saldanha, Gustav Müller-Franzes, Firas Khader, Philip Quirke, Nicholas P. West, Richard Gray, Gordon G.A. Hutchins, Jacqueline A. James, Maurice B. Loughrey, Manuel Salto-Tellez, Hermann Brenner, Alexander Brobeil, Tanwei Yuan, Jenny Chang-Claude, Michael Hoffmeister, Sebastian Foersch, Tianyu Han, Sebastian Keil, Maximilian Schulze-Hagen, Peter Isfort, Philipp Bruners, Georgios Kaissis, Christiane Kuhl, Sven Nebelung, and Jakob Nikolas Kather. "Encrypted federated learning for secure decentralized collaboration in cancer image analysis". In: *Medical Image Analysis* 92 (Feb. 2024), p. 103059.

References V

[17] Soroosh Tayebi Arasteh, Peter Isfort, Marwin Saehn, Gustav Mueller-Franzes, Firas Khader, Jakob Nikolas Kather, Christiane Kuhl, Sven Nebelung, and Daniel Truhn. “Collaborative training of medical artificial intelligence models with non-uniform labels”. In: *Scientific Reports* 13.1 (Apr. 2023).

[18] Soroosh Tayebi Arasteh, Christiane Kuhl, Marwin-Jonathan Saehn, Peter Isfort, Daniel Truhn, and Sven Nebelung. “Enhancing domain generalization in the AI-based analysis of chest radiographs with federated learning”. In: *Scientific Reports* 13.1 (Dec. 2023).

[19] Tian Li, Maziar Sanjabi, and Virginia Smith. “Fair Resource Allocation in Federated Learning”. In: *International Conference on Learning Representations* abs/1905.10497 (2019).

[20] Jianyu Wang, Qinghua Liu, Hao Liang, Gauri Joshi, and H. Poor. “Tackling the Objective Inconsistency Problem in Heterogeneous Federated Optimization”. In: *Neural Information Processing Systems* abs/2007.07481 (2020).

[21] Su Hyeong Lee, Sidharth Sharma, M. Zaheer, and Tian Li. “Efficient Adaptive Federated Optimization”. In: *arXiv.org* abs/2410.18117 (2024).

References VI

[22] Zhiwei Tang and Tsung-Hui Chang. *FedLion: Faster Adaptive Federated Optimization with Fewer Communication*. Tech. rep. IEEE, Apr. 2024, pp. 13316–13320.

[23] Sashank J. Reddi, Zachary B. Charles, M. Zaheer, Zachary Garrett, Keith Rush, Jakub Konecný, Sanjiv Kumar, and H. B. McMahan. "Adaptive Federated Optimization". In: *International Conference on Learning Representations* abs/2003.00295 (2020).

[24] Anit Kumar Sahu, Tian Li, Maziar Sanjabi, M. Zaheer, Ameet Talwalkar, and Virginia Smith. "Federated Optimization in Heterogeneous Networks". In: *Conference on Machine Learning and Systems* (2018).

[25] Rodolfo Stoffel Antunes, Cristiano André da Costa, Arne Küderle, Imrana Abdullahi Yari, and Björn Eskofier. "Federated Learning for Healthcare: Systematic Review and Architecture Proposal". In: *ACM Transactions on Intelligent Systems and Technology* 13.4 (May 2022), pp. 1–23.

[26] Jakub Konecný, H. B. McMahan, Daniel Ramage, and Peter Richtárik. "Federated Optimization: Distributed Machine Learning for On-Device Intelligence". In: *arXiv.org* abs/1610.02527 (2016).

References VII

[27] Jakub Konecný, H. B. McMahan, Felix X. Yu, Peter Richtárik, A. Suresh, and D. Bacon. “Federated Learning: Strategies for Improving Communication Efficiency”. In: *arXiv.org* abs/1610.05492 (2016).

[28] H. B. McMahan, Eider Moore, Daniel Ramage, S.C.D. Hampson, and B. A. Y. Arcas. “Communication-Efficient Learning of Deep Networks from Decentralized Data”. In: *International Conference on Artificial Intelligence and Statistics* (2016), pp. 1273–1282.

[29] Peter Kairouz and H. Brendan McMahan. “Advances and Open Problems in Federated Learning”. In: *Foundations and Trends in Machine Learning* 14.1-2 (June 2021), pp. 1–210.

[30] Jacob Thrasher, Alina Devkota, Prasiddha Siwakotai, Rohit Chivukula, Pranav Poudel, Chaunbo Hu, Binod Bhattarai, and P. Gyawali. “Multimodal Federated Learning in Healthcare: a review”. In: *Journal of Healthcare Informatics Research* abs/2310.09650 (2023).

[31] Alysa Ziying Tan, Han Yu, Lizhen Cui, and Qiang Yang. “Towards Personalized Federated Learning”. In: *IEEE Transactions on Neural Networks and Learning Systems* 34.12 (Dec. 2023), pp. 9587–9603.

References VIII

[32] Francesc Wilhelmi, L. Giupponi, and P. Dini. “Blockchain-enabled Server-less Federated Learning”. In: *arXiv.org* abs/2112.07938 (2021).

[33] Tomas Ortega and Hamid Jafarkhani. “Quantized and Asynchronous Federated Learning”. In: *IEEE Transactions on Communications* 73.4 (Apr. 2025), pp. 2361–2374.